OOAIA Midsem 2026: Problem 2

General Instructions:
Write a single C++ program that acts as a utility toolkit containing four separate tools.
CRITICAL: Your program will be evaluated using an automated grading script. **Do not print any user prompts** (e.g., do not use `cout << "Enter a number: ";`). Your program must read strictly from standard input (`cin`) and print *only* the requested results to standard output (`cout`), followed by a newline.

Part A: The Generic Threshold Filter

Write a standalone template function:
`template <typename T> std::vector<T> filterGreater(const std::vector<T>& data, T threshold);`

This function must iterate through the `data` vector. If an element is strictly greater than the `threshold`, append it to a new vector. Return the new vector.

Part B: Polymorphic Billing Calculator

1. **Base Class:** Create an abstract base class `Service` with a pure virtual function `virtual double calculateCost() const = 0;` and a virtual destructor.
2. **Derived Classes:**
   - Create `HourlyService`. Its constructor takes two `doubles`: `rate` and `hours`. Its cost is `rate * hours`.
   - Create `ProjectService`. Its constructor takes two `doubles`: `fee` and `materials`. Its cost is `fee + materials`.
3. **Calculator:** Write a standard function `double calculateTotalBill(const std::vector<Service*>& services)` that iterates through the vector and returns the sum of all polymorphic service costs.

Part C: The Valley Finder

Write a function: `std::vector<int> findValleys(const std::vector<int>& elevations);`
An element is a "valley" if it is strictly lower than both its immediate left and right neighbors (i.e., `elevations[i] < elevations[i-1]` AND `elevations[i] < elevations[i+1]`).

- Return a vector containing the actual *values* (not the indices) of all valleys.
- The first and last elements can never be valleys.
- If the input vector has fewer than 3 elements, return an empty vector.

Part D: The Particle Collider (Advanced Algorithm)

Write a function: `std::vector<int> collideParticles(const std::vector<int>& particles);`
You are given an array of integers representing particles in a 1D tube.

- The absolute value of the integer represents the **size** of the particle.
- The sign represents the **direction** (positive means moving right, negative means moving left).
- All particles move at the exact same speed.

The Rules of Collision:

1. If two particles collide, the smaller one explodes.
2. If two colliding particles are the exact same size, *both* explode.
3. **The Trap:** Particles moving in the same direction never collide. Furthermore, a negative particle on the left and a positive particle on the right will *never* collide because they are moving away from each other.

Return a vector containing the state of the particles after all collisions are complete.

Execution Engine & Input Format (The `main` function)

Write a `while` loop in your `main()` function that continuously reads an integer `choice` until the number `5` is entered (or the input stream fails). Handle the inputs exactly as follows:

- **If `choice` is 1 (Filter):** Read `N` (size). Read `N` decimal numbers. Read `threshold`. Print filtered numbers separated by a space, followed by a newline.
- **If `choice` is 2 (Billing):** Read `N` (services). For `N` lines: Read `type`. If `1`, read `rate` and `hours`. If `2`, read `fee` and `materials`. Print the total bill, followed by a newline. Ensure you clean up dynamically allocated memory!
- **If `choice` is 3 (Valley Finder):** Read `N` (size). Read `N` integers. Print the valley values separated by a space, followed by a newline.
- **If `choice` is 4 (Collider):** Read `N` (size). Read `N` integers representing the particles. Print the surviving particles separated by a space, followed by a newline. (If all particles explode, print a blank newline).

Sample Input / Output Testing

Standard Input (`cin`):

```
1
5
1.5 5.5 2.0 8.1 3.3
3
2
2
1 50 5
2 1000 200
3
6
10 4 8 2 9 1
4
3
10 2 -5
4
2
8 -8
4
4
-2 -1 1 2
5
```

Standard Output (`cout`):

```
5.5 8.1 3.3
1450
4 2
10

-2 -1 1 2
```